Detecting Meaning Drift in Twitter Sentiment Streams

Informed ageing naive Bayes + an error-based ADWIN detector,
on Kafka, Spark, InfluxDB and Grafana

Group 18 — *member names*

Streaming Data Analysis, Semester 7

Problem: sentiment streams drift

- Tweets are an unbounded stream; topics, vocabulary and opinions change (concept drift).
- A classifier trained once loses accuracy when real-world events change the discussion (Bechini et al., 2021).
- Two kinds of drift:
  - **virtual**: $P(X)$ changes — new words, new topics
  - **real**: $P(y \mid X)$ changes — the *same* words now mean something else
- **Question:** the paper's detector watches the *words*. Does it see changes in *meaning*? If not, can watching the model's own *errors* close the gap?

Base papers

1. **Iosifidis, Oelschlager, Ntoutsi (2017)** — informed ageing MNB on Sentiment140: forget old counts by $e^{-\lambda \Delta t}$, tune λ when a vocabulary detector fires. Our baseline.
2. **Costa, Silva, Antunes, Ribeiro (ICMLA 2014)** — simulate sudden / gradual / recurring drift by re-timestamping real tweets. Our controlled scenarios.
3. **Bechini et al. (IEEE Access 2021)** — event-driven drift in a real Twitter stance stream. Our motivation.

Enhancement: ADWIN (Bifet & Gavaldà, 2007) on the prequential 0/1 error stream, fused with the vocabulary detector.

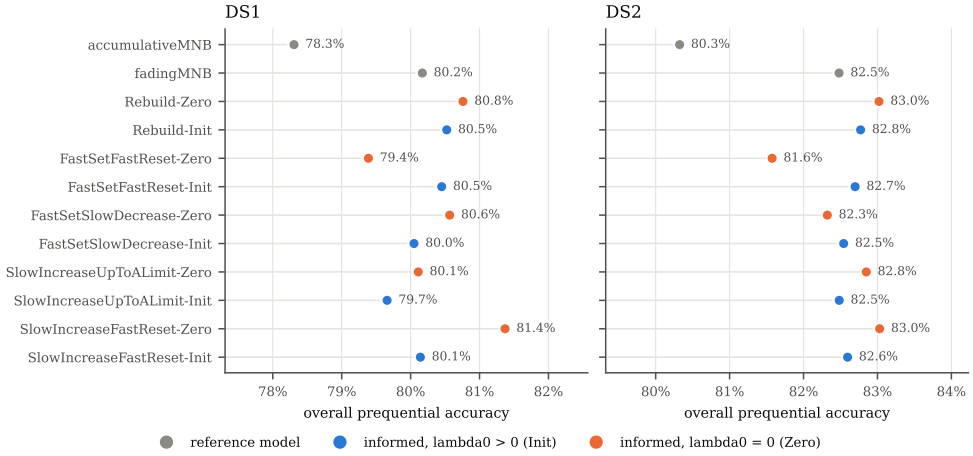
Baseline: informed ageing MNB

- Counts aged *on read*:
  $P(w \mid c) \propto N_{wc}\, e^{-\lambda(t - t_{lo}^{w,c})} + \alpha$, t in **hours**
- Vocabulary detector, every w tweets, per class:
  $\mathrm{precision} = |V_{\text{window}} \cap V_{\text{ref}}| / |V_{\text{ref}}|$
  change if outside $\mu \pm 1.8\sigma$
- 5 reactions: SlowIncrease…, FastSet…, **Rebuild**

Fixes the paper leaves open

- time unit per instance ⇒ model forgets in 100 tweets (68%); per hour works
- accumulated reference vocabulary ⇒ precision drifts down, constant alarms; previous window fixes it
- 10-check history ⇒ false alarms; 20 checks

Reproduction matches the paper's ordering



accumulative worst (78.3 / 80.3%) < fading < informed (up to 81.4 / 83.0%); after DS1's change point adaptive models reach 93–99.9%.

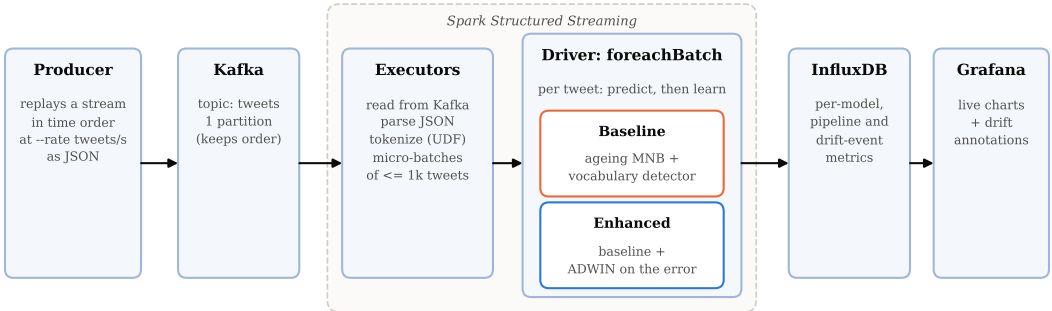
Enhancement: ADWIN on the prediction error

- Prequential: predict, then learn $\Rightarrow$ error $[\hat{y} \neq y]$ for free
- ADWIN ($\delta = 0.01$): cut when the error rate **rises**
- Reaction: **rebuild on ADWIN's post-drift window**
- Fusion: adwin_only, **OR** (main), AND

Learned the hard way

- resetting ADWIN after a rebuild blinded it
- a shared cooldown let one detector's false alarm swallow the other's real alarm
- padding the rebuild to 1k tweets re-learned the old concept (recovery 5.6k $\rightarrow$ 3.0k)

System

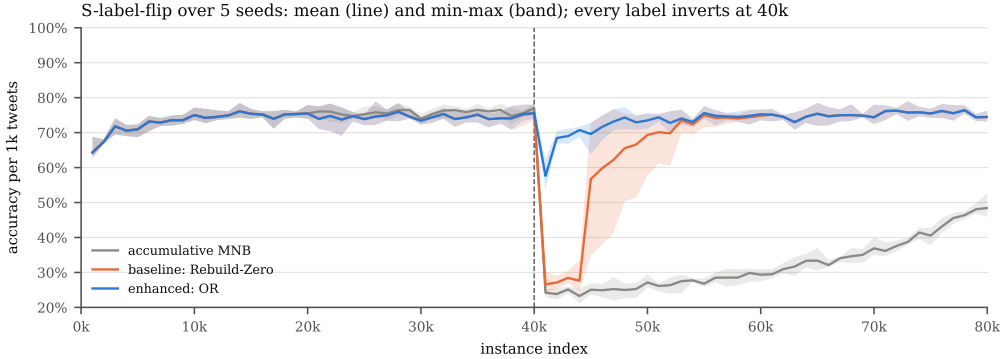


Both models run side by side on identical micro-batches; shared `core/` package with the offline runner.

Streams with known drift points

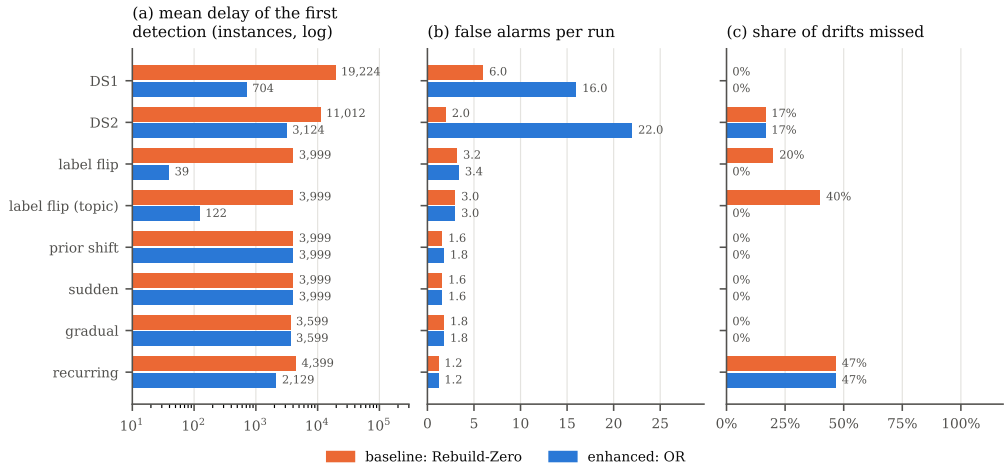
- **DS1**: Sentiment140, 1.6M tweets sorted by time; turns all-negative at 1,324,775
- **DS2**: volatile variant, exactly 1,073,065 tweets (paper's counts)
- **Scenarios** (80k real tweets, 5 seeds): sudden, gradual, recurring topic switch, prior shift 50→10% positive, **label flip** (every label inverts, words unchanged), **label flip on one topic**
- 7 models × 8 streams = **224 runs**; metrics: accuracy, P/R/F1, delay, false alarms, misses, recovery, cost

Headline: label flip (5 seeds)



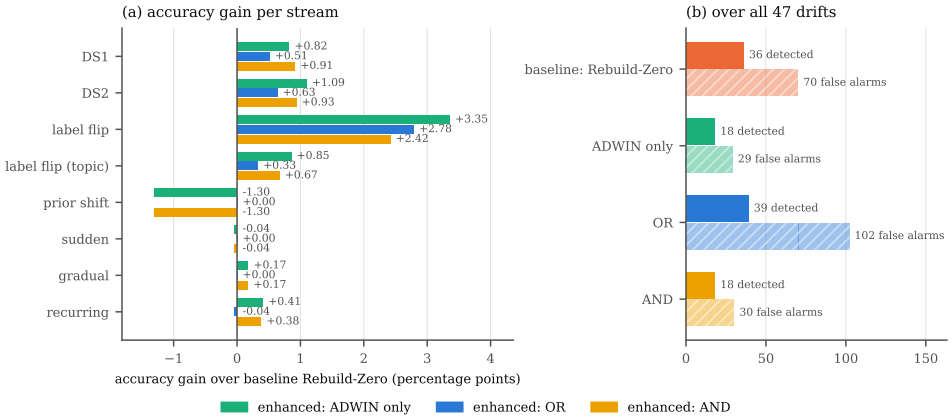
detection **3,999** → **39** tweets recovery **8,204** → **3,183** accuracy **71.0 ± 1.0%** → **73.8 ± 0.3%**

Detection across all streams



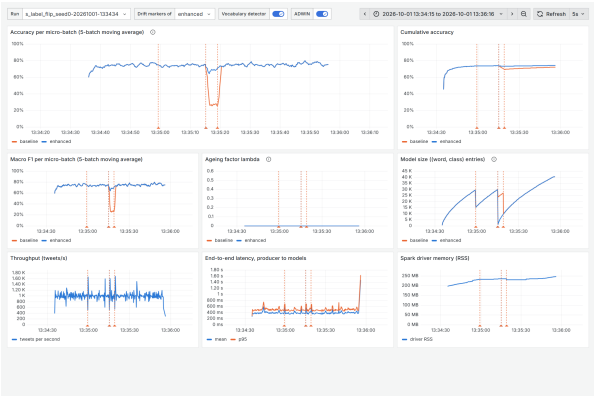
topic-only flip: vocabulary detector misses 2 of 5, ADWIN catches 5 of 5 (~120 tweets); DS1 change point: 19,224 → 704

Real data and fusion modes



DS1 80.8 → 81.3–81.7%, DS2 83.0 → 83.7–84.1%. ADWIN misses prior shifts (no error rise); **OR** keeps both signals: never >0.04 pt below the baseline, ≥ baseline in 28 / 32 stream–seeds.

Live system and cost



- live = offline: 72% vs 74% on the label flip
- 2,000 tweets/s at 0.4 s median latency (1k batches); 5,600 tweets/s with 5k batches
- model time per 1k tweets: **22.6 ms** vs **32.9 ms**; Spark overhead ~170 ms per batch

Takeaways and limitations

- Watch the **errors**, not only the **words**: ADWIN sees meaning drift the vocabulary detector misses or delays (~100× faster on the label flip).
- The detectors are **complementary**: vocabulary → prior / topic shifts; ADWIN → $P(y|X)$ changes.
- Virtual drift barely hurts MNB; λ strategies depend on the time scale, rebuild does not; naive Bayes cannot learn a topic-specific flip (~64% ceiling).
- **Limitations:** noisy emoticon labels, synthetic drifts, DS1/DS2 ground truth only has designed drift points, sequential model on the Spark driver, 2009 data.

Thank you — questions?

Code, data scripts, results: `github.com/CASPER0022/Drift-Aware-Sentiment-Stream`

Backup: Q&A

Why MNB? The base paper's model; one pass, $O(|d|)$ per tweet, interpretable counts that ageing can weight; lets us change *only* the detector.

Why ADWIN, not DDM or Page-Hinkley? Parameter-light (one δ), no fixed window, guarantees on false positives / negatives, and it reports a window width we reuse as the rebuild set.

How is detection delay measured? First detection the model acted on inside $[p, p + T)$ after a known drift point p ($T = 10$k scenarios, 50k real); others are false alarms.

Why Spark if the model is sequential? Spark scales ingestion, parsing and tokenisation and gives exactly-once micro-batches from Kafka; the stateful model runs on the driver. Keyed state per entity would let Spark partition the model too.

How were the drifts made? Real Sentiment140 tweets, topic pools by keywords, balanced per class, re-timestamped into an 80k stream with the drift at a known instance (Costa et al.).